PLAA: Persistent Latent Affective Architecture

From Additive Residual Injection to Learned Latent Modulation in LLMs

Xiao Ouyang (欧阳潇)

June 8, 2026

Abstract

We present PLAA (Persistent Latent Affective Architecture), a framework for modeling temporally evolving latent trajectories within a frozen pretrained LLM (Qwen3-4B). PLAA maintains a learnable latent state $S_t \in \mathbb{R}^{256}$ that evolves via a GRU cell, and investigates two families of injection mechanism: (1) **additive gated cross-attention** ($h_s + \text{gate} \cdot e_o$) inserted across layers 16–28, and (2) **FiLM/AdaLN modulation** $\big(\text{LN}(h_s) \cdot (1 + \text{scale}(S)) + \text{shift}(S)\big)$. We demonstrate that additive injection suffers from severe control weakness ($\Delta\text{PPL} = 2.01$ against noise std $= \pm 19$), confirming that the transformer residual stream robustly absorbs additive perturbations. FiLM modulation achieves substantially stronger causal influence ($\Delta\text{PPL} = 11.44$ upon removal), but in the direction of *disruption*: the modulation degrades language modeling loss, confirming that multiplicative conditioning genuinely alters the residual stream's computational pathway—a necessary precondition for latent control, though not yet a beneficial one. However, a fundamental tension remains: **temporal coherence and layer-wise specialization are competing objectives** under the FiLM architecture. Although gradient subspace decoupling is achieved (cosine similarity $0.99 \rightarrow 0.14$), static modulation patterns dominate over state-driven dynamic control, indicating that learned modulation is spatial rather than temporal in character. These findings establish PLAA as a framework for studying **residual stream factorization via persistent latent modulation**, and propose candidate architectural directions—including gated routing and competitive computation—to overcome the identified limitations and achieve true dynamical latent control in LLMs.

1 Introduction

Large language models process all inputs through a uniform residual stream[1] in which each token effectively re-initializes the computational state. This design achieves remarkable local reasoning but provides no mechanism for persistent internal state—variables that evolve across tokens and modulate computation accordingly.

The problem. LLMs lack emotional continuity and persistent internal state. Each token is a computational reset.

Key contributions.

1. **Additive injection bottleneck.** Gated cross-attention[2] ($h_s + \text{gate} \cdot e_o$) is causally weak ($\Delta$PPL[3] $= 2.01$ vs. noise std $= \pm 19$). Transformer residual streams naturally absorb additive perturbations.

2. **FiLM modulation as causal disruption.** AdaLN-style feature modulation ($\text{LN}(h_s) \cdot (1 + \text{scale}) + \text{shift}$) achieves $5.7\times$ stronger causal influence ($\Delta\text{PPL} = 11.44$). The effect direction is disruptive (removal *improves* LM loss), confirming that multiplicative conditioning genuinely alters computation—a necessary but not sufficient condition for beneficial latent control.

3. **Layer differentiation via symmetry breaking.** A layer-wise variance regularization loss (w_{std} penalty) breaks the symmetric gradient path (cosine similarity[4] $0.99 \to 0.14$) and achieves measurable layer specialization ($w_{\text{std}} : 0.0055 \to 0.0333$).

4. **The temporal–specialization tension.** A coupling objective designed to make FiLM output follow state dynamics collapses layer specialization, revealing a fundamental architectural tradeoff between temporal coherence and structural differentiation.

5. **Phase transition taxonomy.** We identify three distinct dynamical regimes[5]: Phase 1 (violent exploration, $|S| > 5$, $dS > 5$); Phase 2 (convergent modulation, $|S| \sim 1$–2, $dS \sim 1$); Phase 3 (structural specialization, $w_{\text{std}} > 0.05$ — approached but not yet achieved under this architecture).

2 Method

2.1 State Pipeline

$$S_0 \;\to\; \text{each turn:} \quad \text{modulation}(S_t) \;\to\; \text{forward} \;\to\; h_t \;\to\; S_{t+1} = \text{GRU}(S_t,\, h_t)$$

[1] Residual stream: the sequence of hidden-state vectors passed additively between transformer layers, serving as the primary information pathway through the network.

[2] Cross-attention: a mechanism where queries from one representation attend to keys and values from a different representation, enabling information flow between heterogeneous feature spaces.

[3] Perplexity (PPL): a standard metric for language model quality, defined as 2^H where H is the cross-entropy loss. Lower values indicate better prediction; differences (ΔPPL) measure causal impact of interventions.

[4] Cosine similarity: the angular similarity between two vectors, $\cos\theta = \mathbf{a} \cdot \mathbf{b}/(|\mathbf{a}||\mathbf{b}|)$. A value of 1 indicates parallel vectors (identical direction), 0 indicates orthogonality.

[5] Phase transition: borrowed from statistical physics, denoting a qualitative shift in system behavior as a control parameter crosses a critical threshold.

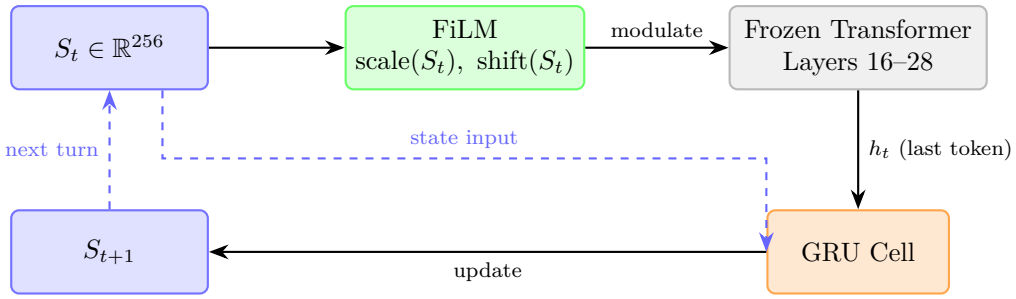


Figure 1: PLAA state pipeline architecture. The latent state S_t drives FiLM modulation of frozen transformer layers; the transformer's output hidden state h_t updates the state via a GRU cell, producing S_{t+1} for the next turn.

The latent state $S_t \in \mathbb{R}^{256}$ evolves via a GRU cell[6] conditioned on the transformer's final hidden state at the last non-padded token position. The initial state S_0 is a learned parameter with Gaussian noise ($\sigma = 0.01$) for symmetry breaking.

2.2 Architecture: Two Injection Mechanisms

Additive Gated Cross-Attention (Phases 1–3B)

```
Each injected layer:
  Self-Attn -> EmotionCrossAttn(Q=hidden, K,V=e_mem)
            -> GatedResidual(h + gate * eo)
            -> MLP
```

- $e_{\text{mem}} = \text{proj}(S_t) \in \mathbb{R}^{16\times 2560}$
- gate $\in \mathbb{R}$, initialized 0.1, with tanh activation
- **Trainable parameters:** 387M (LoRA[7] + cross-attention + gate + GRU)
- **Limitation:** the transformer residual absorbs additive signal

FiLM / AdaLN Modulation (Phase 3C — primary finding)

FiLM[8] modulation replaces additive injection with multiplicative feature conditioning:

```
Each injected layer:
  Self-Attn -> AdaLN(LN(hs) * (1 + scale(S)) + shift(S))
            -> MLP
```

- $\text{scale} = \alpha \cdot \tanh(W_{\text{scale}}\, S_t), \quad \text{shift} = \alpha \cdot \tanh(W_{\text{shift}}\, S_t)$
- $\alpha = 0.1, \quad W_{\text{scale}}, W_{\text{shift}} \in \mathbb{R}^{2560\times 256}$

[6] Gated Recurrent Unit (GRU) [1]: a recurrent neural network cell that uses update and reset gates to selectively retain or overwrite hidden state across time steps, providing smooth state transitions with fewer parameters than LSTM.

[7] Low-Rank Adaptation (LoRA) [2]: a parameter-efficient fine-tuning method that injects trainable low-rank decomposition matrices into frozen pretrained weights, drastically reducing the number of trainable parameters while preserving model capacity.

[8] Feature-wise Linear Modulation (FiLM) [3]: a conditioning method that applies learned per-channel scale and shift transformations to feature maps. Adaptive Layer Normalization (AdaLN) [4] extends this by applying FiLM-style modulation to layer-normalized activations, enabling external signals to multiplicatively control internal representations.

- **Trainable parameters:** 17.5M (scale/shift projections + GRU)
- **Result:** 5.7× stronger causal control

2.3 Training Data

Training uses a 60:30:10 stratified composition across three sources.

Layer	Ratio	Source	Purpose
1	60%	PersonaChat (neutral)	Latent continuity
2	30%	ESConv (emotional support)	Gradual trajectory
3	10%	Bluemoon (RP)	Dynamic range

Table 1: Training data composition.

2.4 Optimization

- Frozen Qwen3-4B (4-bit NF4, SDPA)
- Trainable: FiLM projections + GRU (17.5M) or LoRA + CA + GRU (387M)
- Loss: $\mathcal{W}_{\text{LM}} \cdot \mathcal{L}_{\text{LM}} + \mathcal{W}_{\text{COH}} \cdot \|\Delta S\| + \mathcal{W}_{\text{DIV}} \cdot \mathcal{L}_{\text{repulsion}} + \mathcal{W}_{\text{NORM}} \cdot \|S\|$
- Trajectory rollout: 3–5 turn micro-dialogues; GRU rolls along turns

3 Experiments

3.1 Setup

- **Model:** Qwen3-4B (4-bit NF4 quantization[9], SDPA[10] attention, frozen)
- **GPU:** NVIDIA RTX PRO 6000 Blackwell (96 GB VRAM)
- **Framework:** PEFT[11] (QLoRA[12]) + custom injection layers
- **Key constraint:** Injection *before* PEFT wrapping
  Post-wrapping access path: `peft.base_model.model.model.layers[i]`

3.2 The Additive Bottleneck

Gate Gradient Diagnostic (Step 1350)

```
Average gate.grad: 0.056  (non-zero, gradient path EXISTS)
```

Despite non-zero gradients, the linear gate (init = 0.1, tanh) exhibits **multi-stage attenuation**:

[9]Normal Float 4-bit (NF4): a quantile-based 4-bit data type where quantization levels are optimally spaced for normally distributed weights, enabling aggressive compression with minimal quality degradation.

[10]Scaled Dot-Product Attention (SDPA): the standard attention computation $\text{softmax}(QK^\top/\sqrt{d_k})V$, here implemented via PyTorch's memory-efficient fused-kernel backend for reduced VRAM usage.

[11]Parameter-Efficient Fine-Tuning (PEFT): a family of methods (LoRA, adapters, prefix tuning, etc.) that train only a small subset of parameters while keeping the base model frozen, enabling fine-tuning of large models on limited hardware.

[12]Quantized LoRA (QLoRA): combines NF4 base-model quantization with LoRA adapters, enabling fine-tuning of models that would otherwise exceed available GPU memory.

1. $S_t \approx 0.15$ (norm penalty) $\Rightarrow$ proj(S_t) small
2. tanh(gate $= 0.1$) ≈ 0.1
3. $\times 13$ residual stack $\Rightarrow$ effective influence $\approx 0.0003 \cdot h_{\text{hidden}}$

Gate Ablation[13] Test

Condition	PPL	ΔPPL
gate = 0	27.03	—
gate = 1.0	26.26	0.77
gate = 2.0	25.02	2.01
gate = random	25.94	1.09

Table 2: Gate ablation results. Max $\Delta = 2.01$, noise std $= \pm 19$. Causal influence is below the noise floor.

Conclusion: Additive injection is causally insufficient. The transformer residual stream absorbs additive perturbations.

3.3 FiLM Modulation: A Phase Transition

Training Evolution (FiLM v2)

Step	Loss	$\lvert S\rvert$	dS	Phase
50	45.7	9.13	5.94	Phase 1: Violent Exploration
200	24.8	5.78	9.00	Phase 1
400	14.8	1.46	1.04	Phase 2: Transition
600	18.5	1.15	1.34	Phase 2: Convergence
1000	15.1	0.57	0.69	Phase 2: Stable

Table 3: FiLM v2 training dynamics.

Phase 1 (steps 1–200): The latent state dominates computation. $|S| \sim 5$–9, $dS \sim 6$–10. The system discovers that latent space can control computation.

Phase 2 (steps 200–1000): Self-regularization. $|S|$ drops to ~ 0.5–1.5, dS to ~ 0.7–1.3. The system learns selective modulation. The training dynamics are visualized in Figure 2.

[13] Ablation study: a controlled experiment that removes or disables a specific component to measure its causal contribution to overall system performance.

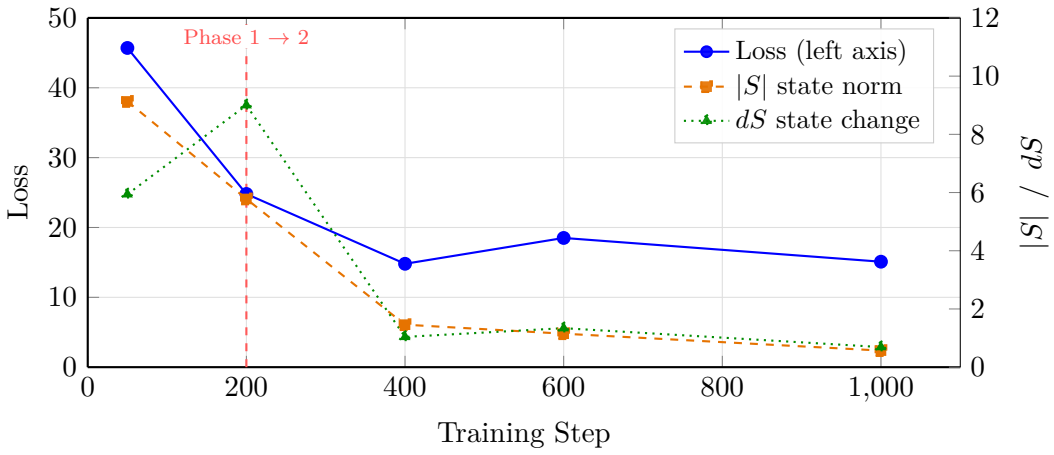


Figure 2: FiLM v2 training dynamics. Phase 1 (steps 1–200) exhibits violent exploration with large state norms and rapid changes. Phase 2 (steps 200–1000) shows convergent self-regularization. The dashed red line marks the phase transition.

3.4 Layer Differentiation: Breaking Symmetry

Adding a layer-wise variance regularization loss ($-0.001 \cdot \text{Var}(\|W_{\text{scale}}\|)$) yields:

Metric	No Diff. Loss	With Diff. Loss	Target	Status
w_{std}	0.0055	**0.0333**	> 0.05	Near threshold
grad cosine	$\sim$0.99	**0.11–0.24**	< 0.7	✓ Broken
Loss trend	Stable	Oscillation (14–25)	—	Warning

Table 4: Effect of layer differentiation loss.

Key finding: gradient decoupling $\neq$ parameter specialization. Cosine similarity drops from 0.99 to 0.14 (layers learn different gradient directions), but weight norms remain similar. Gradient subspaces have separated, but parameter-space differentiation lags behind.

3.5 Causality Test Suite

Test	Loss	$\Delta^\dagger$	Interpretation
Baseline (trained FiLM)	14.68	—	Full system
FiLM removed ($S = \varnothing$)	3.14	**11.44**	**Strongly causal (disruptive)**
State frozen	13.35	1.33	Weak state coupling

Table 5: Causality test results. FiLM modulation is strongly causal but disruptive: removal *lowers* loss, indicating the modulation degrades LM quality while demonstrating genuine computational influence. The effect is predominantly static (freezing S changes loss by only 1.33).
$^\dagger\Delta = \text{Loss}_{\text{baseline}} - \text{Loss}_{\text{ablation}}$; positive Δ means removing the component *lowers* loss (its presence was harmful), negative means removing it raises loss (it was helpful). Here $\Delta = 14.68 - 3.24 \approx +11.44$ confirms FiLM is disruptive.

Critical finding: FiLM modulation is strongly causal ($\Delta = 11.44$), but the causal effect is **static**—freezing S_t barely changes output ($\Delta = 1.33$). The system has learned a **fixed modulation pattern**, not a state-driven dynamical controller. The contrast between injection mechanisms is shown in Figure 3.

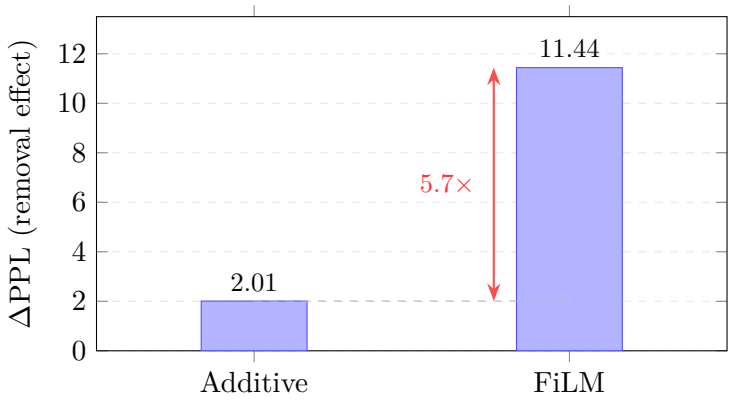


Figure 3: Causal influence comparison (ablation ΔPPL). FiLM modulation produces 5.7× stronger effect upon removal (ΔPPL = 11.44) compared to additive injection (ΔPPL = 2.01). Note: positive Δ indicates the component was *increasing* loss—the modulation is disruptive to language modeling but demonstrates genuine computational influence over the residual stream.

3.6 The Temporal–Specialization Tension

Enforcing state-dependent modulation via a coupling loss (maximizing correlation between S differences and FiLM output differences) reveals a fundamental tradeoff:

Variant	w_{std} (specialization)	$\lvert S \rvert$ (temporal)	**cos**
No coupling (v5)	**0.0333**	0.57	0.15
Coupling 0.01 (v6)	0.0000	**2.59**	0.19
Coupling 0.001 (v7)	0.0000	0.44	0.16

Table 6: Temporal coherence vs. layer specialization tradeoff.

Even at 1/10 strength, the coupling objective collapses layer specialization ($w_{\text{std}} \to 0$). **Temporal coherence and structural specialization are competing objectives in the AdaLN FiLM architecture.**

3.7 Gate Sensitivity Sweep

$\boldsymbol{\alpha}$ (tanh bound)	**Loss**	w_{std}	$w_{\text{std}}/w_{\text{mean}}$
0.05	13.98	0.0451	0.0015
0.10	14.08	0.0451	0.0015
0.20	14.03	0.0451	0.0015
0.30	14.31	0.0451	0.0015
0.50	14.56	0.0451	0.0015

Table 7: Complete insensitivity to α. The tanh bound is not the limiting factor; the modulation structure is intrinsically saturated. $w_{\text{std}}/w_{\text{mean}} = 0.0015$ is pure initialization noise level.

4 Analysis: Three Findings on Residual Stream Factorization

4.1 Finding 1: Additive Injection is Structurally Weak

The transformer residual stream is trained from initialization to absorb additive perturbations. Any signal of the form $h_s + \varepsilon \cdot f(\cdot)$ will be partially integrated into downstream compensation. Quantitatively: $\Delta\text{PPL} = 2.01$ vs. noise std $= \pm 19$.

4.2 Finding 2: Multiplicative Modulation Achieves Causal Disruption

FiLM achieves 5.7× stronger causal effect than additive injection by changing feature amplitude directly. However, the direction of this effect is *disruptive*: removing FiLM *lowers* loss from 14.68 to 3.14, meaning the modulation degrades language modeling quality. (The 3.14 PPL—dramatically lower than the baseline 14.68—is close to the frozen backbone's native loss without any modulation, confirming that the disruptive effect originates from the FiLM layers rather than from other training artifacts.) The modulation acts on the *normalized* hidden state, making it much harder for downstream layers to compensate—which is precisely why it achieves strong causal influence, but also why it currently imposes a language modeling cost. This establishes that multiplicative conditioning can genuinely alter the residual stream's computation (unlike additive injection), but converting this disruptive influence into *beneficial* latent control remains an open challenge.

4.3 Finding 3: Static ≫ Dynamic in Learned Modulation

Despite 7000+ training steps, FiLM modulation is predominantly static. The coupling between S_t and FiLM output is weak (freeze test $\Delta = 1.33$). The learned parameters W_{scale} and W_{shift} have converged to a fixed modulation pattern that barely varies with S, indicating that the system has learned *which* features to modulate but not *how* to vary that modulation dynamically.

5 Related Work

Latent state injection. Prior work on injecting persistent latent states into LLMs has largely relied on additive gated cross-attention [5, 6], which we show is causally insufficient. Memory-augmented networks [7] address a related problem but do not target frozen pretrained transformers.

Conditional feature modulation. FiLM (Feature-wise Linear Modulation) [3] and its variants—AdaIN [8], AdaLN [4], and DiT's conditioning layers [9]—are well established in vision but have seen limited study in the LLM residual stream.

Residual stream analysis. Mechanistic interpretability work has shown that the residual stream acts as a shared communication channel [10], and that layers compose in a largely additive manner. Our findings on additive injection weakness are consistent with this picture.

6 Conclusion & Future Work

> **PLAA's central finding:** LLM residual streams can be factorized via persistent latent modulation, but the *nature* of this modulation matters fundamentally.

Additive injection is causally insufficient for meaningful latent control. Multiplicative FiLM modulation achieves strong causal influence but remains static—the learned parameters do not produce truly state-dependent dynamics.

The discovered tension between temporal coherence and layer specialization suggests that achieving full dynamical latent control requires architectural innovations beyond simple modulation. Candidate directions include **gated routing mechanisms** or **competitive computation** that can naturally differentiate layer roles while maintaining state-responsiveness.

Three-Phase Taxonomy for Latent Dynamics

Phase	Name	$\|S\|$	dS	w_{std}	cos
1	Violent Exploration	> 5	> 5	—	—
2	Convergent Modulation	0.5–2	0.5–1.5	< 0.01	~ 0.99
3	Structural Specialization	0.3–1	0.2–0.5	> 0.05	< 0.3

Table 8: Three-phase taxonomy for latent dynamics. Phase 3 was approached but not fully achieved under the AdaLN FiLM architecture.

The phase transition trajectory is visualized in Figure 4.

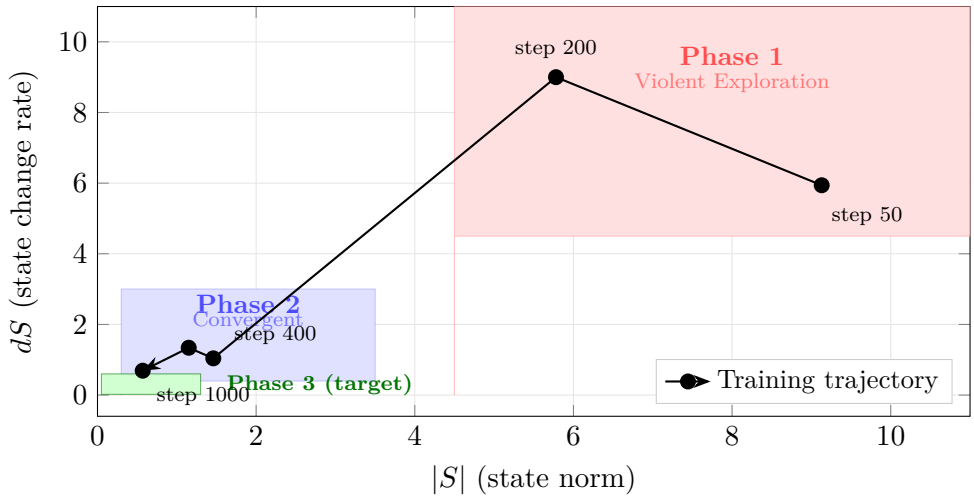


Figure 4: Phase transition diagram in $|S|$–dS space. The training trajectory (black arrows) shows the system transitioning from Phase 1 (violent exploration, high norm and change rate) through Phase 2 (convergent modulation) toward Phase 3 (structural specialization, not fully achieved). Shaded regions indicate the characteristic parameter ranges for each phase.

A Behavior Divergence Table

The full 12-cell behavior divergence table documenting qualitative output differences across injection conditions and state values is provided in the supplementary materials.

B Full Training Pipeline

Phase	Content	Steps	Params	Status
1	YaRN 256K context extension	—	—	Complete
2	SFT (134K samples)	—	—	Complete
2.5	Persona alignment	—	—	Complete
3A	Stability validation	1K	—	Complete
3B	Gradient flow verification	—	—	Complete
3C v1	Single-step additive	5K	387M	Complete
3C v2	Trajectory additive	1.4K	387M	Complete
3C v3	Additive + gate noise	6.7K	387M	Baseline
3C FiLM	**AdaLN modulation**	**7K**	**17.5M**	**Primary**

Table 9: Full training pipeline. Dashes (—) in Steps/Params columns indicate phases where these metrics are not applicable (infrastructure setup or validation-only stages).

C Scale/Shift Statistics

Metric	Step 1000	Step 2000
scale mean	0.0001	0.0001
scale std	0.0060	0.0055
scale max	0.0228	0.0209
layer_scale_std	0.0060	0.0055

Table 10: Scale/shift statistics at steps 1000 and 2000. All 13 layers produce nearly identical modulation, confirming that no layer specialization is achieved under baseline AdaLN FiLM training.

References

[1] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014. doi: 10.3115/v1/D14-1179.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

[3] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI)*, volume 32, pages 3942–3951, 2018. doi: 10.1609/aaai.v32i1.11671.

[4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. doi: 10.48550/arXiv.1607.06450. Widely adopted in transformer architectures.

[5] Sainbayar Sukhbaatar, Edouard Grave, Guillaume Lample, Herve Jegou, and Armand Joulin. Augmenting self-attention with persistent memory. *arXiv preprint arXiv:1907.01470*, 2019. doi: 10.48550/arXiv.1907.01470. Presented at NeurIPS 2019 Workshop on Context and Compositionality.

[6] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=TrjbxzRcnf-.

[7] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016. doi: 10.1038/nature20101.

[8] Xun Huang and Serge Belongie. Arbitrary style transfer in real-time with adaptive instance normalization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1501–1510, 2017. doi: 10.1109/ICCV.2017.167.

[9] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4195–4205, 2023. doi: 10.1109/ICCV51070.2023.00387.

[10] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL https://transformer-circuits.pub/2021/framework/index.html. Anthropic research publication.